Tracing length of a string using recursion.

suppos string = "Ball"

1st function call

find_length(user_str) # "Ball"

└ "Ball" == " " ? False

└ return 1 + find_length(user_str [1:]) #Final output

waiting ← # return 1 + find_length("all") 1+3 = 4

2nd find_length ("all")

└ "all" = " " ? False

waiting ← └ return 1 + find_length("ll") 1+2 = 3

3rd find_length ("ll")

└ "ll" == " " ? False

waiting ← └ return 1 + find_length("l") 1+1 = 2

4th find_length("l")

└ "l" == " " ? False

waiting ← └ return 1 + find_length(" ") 1+0 = 1

5th find_length(" ")

└ " " == " " ? True

└ return 0

Tracing factorial of a number using recursion.
factorial (number) # suppose number is 5

1^{st} Function call
factorial (5)
— number = 5
— 5 = = 1 ? False
waiting ←— return 5 * factorial(5-1) # 4
└ waits because another function call

Result
5×4 = 120 //

2nd factorial (4)
└ number = 4
4 = = 1 ? False
waiting ←— return 4 * factorial (4 -1) # 3

4 * 6 = 24

3rd factorial (3)
└ number = 3
└ 3 = = 1 ? False
waiting ←— └ return 3 * factorial (3-1) # 2

3 * 2 = 6

4th factorial (2)
└ number = 2
└ 2 = = 1 ? False
waiting ←— └ return 2 * factorial (2-1) # 1

2 * 1 = 2

5th factorial (1)
returns 1
└ number = 1
└ 1 = = 1 : True
return 1 # 1